Reviewer Pack — Minimal Rank of Tropical Curves and Communication Complexity…

Entry 92f8c4dcf484 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 22:38:24 UTC

Minimal Rank of Tropical Curves and Communication Complexity Rank Correlation

Entry ID: `92f8c4dcf484` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 22:38:24 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity

Statement:

> The rank of the minimal tropical curve associated with a given communication complexity matrix M is linearly correlated with the rank of M, such that rank(tropical_curve(M)) = Θ(rank(M)). For all instances with property P, if property Q holds for M, then property R holds for the corresponding tropical curve.

Rationale (proposer's reasoning):

> Tropical geometry provides a framework to study polynomial equations over the max-plus semiring. This conjecture leverages the idea that the structure of communication complexity matrices can be understood through their associated tropical curves, possibly revealing hidden connections between geometric and algorithmic properties.

Taxonomy category: `communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8ee4d4757d1ba705`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given communication complexity matrix M with rank n ≤ 40, if the correlation coefficient between the ranks of M and its associated tropical curve is greater than or equal to 0.5 for at least 80% of seeds, support for the conjecture is provided.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"tropical geometry" AND "communication complexity" AND minimal rank` - `"tropical curve" related to communication complexity matrix rank` - `"property P in communication complexity implies property Q in tropical curve"`

Top relevant hits considered: - [s2:10.1017/bsl.2018.13] 2017 EUROPEAN SUMMER MEETING OF THE ASSOCIATION FOR SYMBOLIC LOGIC LOGIC COLLOQUIUM '17 Stockholm, Sweden August 14–20,

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = max(range(i, n), key=lambda k: abs(matrix[k][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        pivot = matrix[i][i]
        for j in range(i + 1, n):
            factor = Fraction(matrix[j][i], pivot)
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

def tropical_rank(M):
    n = len(M)
    T = [[max(row[j], col[j]) for j in range(n)] for row in M]
    return gaussian_elimination(T)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 30
    correlation_sum = 0.0
    support_count = 0

    for _ in range(instances_tested):
```

```
        M = [[random.randint(0, 10) for _ in range(n)] for _ in range(n)]
        rank_M = gaussian_elimination(M)
        rank_T = tropical_rank(M)
        correlation_sum += abs(rank_M - rank_T) / (rank_M + rank_T)

    mean_correlation = correlation_sum / instances_tested
    support_fraction = support_count / instances_tested

    return {
        "metric_name": "correlation",
        "metric_value": mean_correlation,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": mean_correlation >= 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff33acc0.py", line 67, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff33acc0.py", line 44, in run_trial
    M = [[random.randint(0, 10) for _ in range(n)] for _ in range(n)]
                                                                 ^
NameError: name 'n' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed due to an undefined variable ‘n’, which prevented the execution of the test and the production of data necessary to evaluate the conjecture. | next: Define the variable ‘n’ in the test code before running the trial, ensuring that all required variables are properly initialized.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13958
2	propose	ollama_remote	glm4:latest	0	0	12342
3	preregistration	ollama_remote	glm4:latest	0	0	11302
4	novelty	ollama_remote	glm4:latest	0	0	8528
5	novelty	ollama_remote	glm4:latest	0	0	9296
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19099
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10041
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10198
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7914
10	judge	ollama_remote	glm4:latest	0	0	20157

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 122835 ms total latency. Provider mix: {‘ollama_remote’: 10}

(full prompt+response transcripts available in `research/audit/92f8c4dcf484.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/92f8c4dcf484.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/92f8c4dcf484.tar.gz` (if generated)